EventBusClient

v. 0.1.0

Nguyen Huynh Tri Cuong

09.07.2025

Contents

Chapter 1

Introduction

The component **EventBusClient** provides a modular, high-level messaging framework built on top of the message broker **RabbitMQ**. It abstracts low-level standard protocol operations and adds:

- Plugin support (custom serializers, handlers, message types)
- Dynamic configuration
- Reconnect and lifecycle management

It enables multiple projects to reuse a consistent event bus API without rewriting **RabbitMQ** logic.

The **EventBusClient** is part of a test automation framework called **RobotFramework AIO**, but can also be installed and used stand-alone.

RabbitMQ is an open-source message broker that enables different parts of an application or separate applications to communicate with each other by sending and receiving messages asynchronously. It acts as an intermediary between producers (which send messages) and consumers (which receive and process messages). **RabbitMQ** decouples the sender and receiver, allowing them to operate independently and at their own pace.

Messages are stored in queues until they can be processed, ensuring reliability even if a consumer is temporarily unavailable.

RabbitMQ terms

Term	Description
Producer	Application or service that sends messages
Consumer	Application or service that receives and processes messages
Queue	Buffer that temporarily stores messages
Exchange	Routes messages from producers to queues based on routing rules
Binding	Link between an exchange and a queue, defining how messages are routed
Routing Key	Address-like string used to decide how messages are routed
AMQP	Advanced Message Queuing Protocol, the standard protocol **RabbitMQ** uses

For more detailed information about these components and framworks please refer to the corresponding homepages.

Chapter 2

Description

2.1 EventBusClient

2.1.1 Overview

The **EventBusClient** provides a high-level API for interacting with a RabbitMQ-based event bus system. It abstracts low-level AMQP details and supports dynamic loading of project-specific plugins for serializers, exchange handlers, and message structures. This client is designed for modularity and ease of use across multiple projects.

—

2.1.2 Features

- **Dynamic plugin loading**: Load custom serializers, exchange handlers, and message classes at runtime from a configurable plugins directory.
- **Thread-safe publishing**: Supports safe publishing from multiple threads with minimal locking overhead.
- **Automatic reconnect**: Automatically reconnects to RabbitMQ after connection loss (if enabled in configuration).
- **Pluggable architecture**: Supports both built-in and project-specific extensions.
- **Supports JSON, Protobuf, Pickle serializers**.

2.1.3 Architecture

The EventBusClient is built around a modular architecture that allows for easy extension and customization. It consists of the following components:

- **Base Classes**: Define interfaces for exchange handlers, serializers, and messages.
- **Plugins Directory**: Contains custom implementations of the base classes.
- **Configuration File**: Specifies the plugins path, RabbitMQ connection details, and selected implementations.
- **EventBusClient Class**: The main class that interacts with RabbitMQ and manages plugins.
- **Message Classes**: Define the structure of messages sent and received over the event bus.
- **Exchange Handlers**: Handle the declaration and publishing of messages to RabbitMQ exchanges.
- **Serializers**: Convert messages to and from byte streams for transmission over RabbitMQ.

The client uses a combination of built-in plugins and user-defined plugins to provide flexibility in message handling and serialization.

2.1.4 Installation

To install the EventBusClient, use pip to install the package from PyPI:

```
pip install event-bus-client
```

Ensure you have RabbitMQ server running and accessible at the specified host and port in the configuration file.

2.1.5 Dependencies

The EventBusClient requires the following dependencies:

- **pika**: For RabbitMQ communication.
- **protobuf**: For Protocol Buffers serialization (if using ProtobufSerializer).
- **jsonpickle**: For JSON serialization (if using JSONSerializer).
- **pickle**: For Pickle serialization (if using PickleSerializer).

Ensure these dependencies are installed in your Python environment. You can install them using pip:

```
pip install pika protobuf jsonpickle
```

2.1.6 Directory Structure

```
project/
|-- EventBusClient/
|   |-- event_bus_client.py
|   |-- connection.py
|   |-- plugin_loader.py
|   |-- publisher.py
|   |-- subscriber.py
|   |-- qlogger.py
|   |-- message/
|   |   |-- base_message.py
|   |-- exchange_handler/
|   |   |-- base.py
|   |   |-- xr_topic_handler.py
|   |-- serializer/
|   |   |-- base_serializer.py
|   |   |-- json_serializer.py
|   |   |-- protobuf_serializer.py
|   |   |-- pickle_serializer.py
|   |-- plugins/
|   |   |-- mcpi/
|   |   |   |-- serialize/
|   |   |   |-- message/
|   |   |   |-- exchange/
|   |-- config/
|   |   |-- config.jsonp
|-- app.py
```

2.1.7 Configuration

The `config.jsonp` file controls all aspects of the EventBusClient, including plugin paths, RabbitMQ connection, and selected implementations.

```
{
  "plugins_path": "./plugins",
  "host": "localhost",
  "port": 5672,
```

```
  "serializer": "PickleSerializer",
  "exchange_handler": "XRTopicExchangeHandler",
  "message_class": "ListenerEventMsg",
  "threadsafe_publish": true,
  "auto_reconnect": true,
  "qos_prefetch": 10
}
```

—

2.1.8 Usage

Initialization

Create an EventBusClient instance from configuration:

```
[language=Python]
from event_bus.client import EventBusClient

client = await EventBusClient.from_config("./config/config.jsonp")
```

—

Publishing Messages

Send a message using the configured serializer and exchange handler:

```
[language=Python]
msg = ListenerEventMsg(event="start_cycle", timestamp=1234567890)
await client.send("zone1.topic.start", msg)
```

Enable thread-safe publish from a non-async thread:

```
[language=Python]
client.send("zone1.topic.alert", msg, threadsafe=True)
```

—

Subscribing to Messages

Subscribe to a routing key and handle incoming messages:

```
[language=Python]
async def on_message(msg: ListenerEventMsg):
    print(f"Received event: {msg.event} at {msg.timestamp}")

await client.on("zone1.#", ListenerEventMsg, on_message)
```

—

2.1.9 Example: Producer and Consumer

This example (from `test.py`) starts a producer and a consumer in separate processes:

```
[language=Python]
# Start consumer
consumer = Process(target=run_process, args=(consumer_process, config_path))
consumer.start()

# Start producer
```

```
producer = Process(target=run_process, args=(producer_process, config_path))
producer.start()
# Wait for both to finish
producer.join()
consumer.join()
```

2.1.10 Example: Custom Plugin Structure

To create a custom plugin, follow these steps:

- Create a directory for your plugin in the configured plugins path.
- Implement the required interfaces (e.g., BaseExchangeHandler, BaseMessage, BaseSerializer).
- Register your plugin in the `config.jsonp` file.
- Ensure your plugin is discoverable by the EventBusClient.
- Place your plugin code in the plugins directory, following the structure:
- For example, if your plugin is named `CustomPlugin`, the directory structure should look like this:
- ```
  plugins/
  |-- CustomPlugin/
      |-- __init__.py
      |-- custom_exchange_handler.py
      |-- custom_message.py
      |-- custom_serializer.py
  ```

2.1.11 Example: Built-in Plugins

The EventBusClient comes with several built-in plugins that can be used directly or extended:

- **XRTopicExchangeHandler**: Handles topic exchanges with RabbitMQ.
- **ListenerEventMsg**: A default message class for listener events.
- **PickleSerializer**: Serializes messages using Python's Pickle module.
- **JSONSerializer**: Serializes messages to JSON format.
- **ProtobufSerializer**: Serializes messages using Protocol Buffers.

These plugins can be used as-is or extended to create custom functionality.

2.1.12 Example: Built-in Configuration

The built-in configuration file `config.jsonp` provides a starting point for configuring the EventBusClient:

```
{
  "plugins_path": "./plugins",
  "host": "localhost",
  "port": 5672,
  "serializer": "PickleSerializer",
  "exchange_handler": "XRTopicExchangeHandler",
  "message_class": "ListenerEventMsg",
  "threadsafe_publish": true,
  "auto_reconnect": true,
  "qos_prefetch": 10
}
```

This configuration specifies the plugins path, RabbitMQ connection details, serializer, exchange handler, message class, and other options.
```

2.1.13 Example: Custom Exchange Handler

To create a custom exchange handler, implement the required interface and place it in the plugins directory. For example, a custom exchange handler might look like this:

```
[language=Python]
from event_bus.plugins import BaseExchangeHandler
class CustomExchangeHandler(BaseExchangeHandler):
    def declare_exchange(self, channel):
        # Custom exchange declaration logic
        pass

    def publish(self, channel, routing_key, message):
        # Custom publish logic
        pass
# Register the plugin in config.jsonp
{
  "plugins_path": "./plugins",
  "exchange_handler": "CustomExchangeHandler"
}
```

2.1.14 Example: Custom Message Class

To create a custom message class, define it in your project and register it in the configuration:

```
[language=Python]
from event_bus.plugins import BaseMessage
class CustomMessage(BaseMessage):
    def __init__(self, data):
        self.data = data

    @classmethod
    def from_data(cls, data):
        pass

    @abstractmethod
    def get_value(self):
        pass
# Register the message class in config.jsonp
{
  "plugins_path": "./plugins",
  "message_class": "CustomMessage"
}
```

2.1.15 Example: Custom Serializer

To create a custom serializer, implement the required interface and place it in the plugins directory. For example, a custom serializer might look like this:

```
[language=Python]
from event_bus.plugins import BaseSerializer
class CustomSerializer(BaseSerializer):
    def serialize(self, obj):
        # Custom serialization logic
        pass

    def deserialize(self, data):
        # Custom deserialization logic
        pass
# Register the plugin in config.jsonp
{
```

```
  "plugins_path": "./plugins",
  "serializer": "CustomSerializer"
}
```

Chapter 3

EventBusClient.py

3.1 Class: CEventBusClient

Imported by:

```
from EventBusClient.EventBusClient import CEventBusClient
```

TODO

3.1.1 Method: getVersion

Returns the version of EventBusClient as string.

3.1.2 Method: getVersionDate

Returns the version date of EventBusClient as string.

Chapter 4

connection.py

4.1 Class: ConnectionManager

Imported by:

```
from EventBusClient.connection import ConnectionManager
```

ConnectionManager: Manages RabbitMQ connections, channels, and exchanges.

4.1.1 Method: register_exchange_handler

Register an exchange handler to handle messages from the exchange.

Arguments:

- `handler`

  / *Condition*: required / *Type*: ExchangeHandler /
  The exchange handler to register. It should be an instance of ExchangeHandler or its subclasses.

4.1.2 Method: unregister_exchange_handler

Unregister an exchange handler.

Arguments:

- `handler`

  / *Condition*: required / *Type*: ExchangeHandler /
  The exchange handler to unregister. It should be an instance of ExchangeHandler or its subclasses.

Chapter 5

event_bus_client.py

5.1 Class: EventBusClient

Imported by:

```
from EventBusClient.event_bus_client import EventBusClient
```

EventBusClient: Client for interacting with the event bus system.

Chapter 6

base.py

6.1 Class: ExchangeHandler

Imported by:

```
from EventBusClient.exchange_handler.base import ExchangeHandler
```

Chapter 7

topic_handler.py

7.1 Class: TopicExchangeHandler

Imported by:

```
from EventBusClient.exchange_handler.topic_handler import TopicExchangeHandler
```

Chapter 8

x_rtopic_handler.py

8.1 Class: XRTopicExchangeHandler

Imported by:

```
from EventBusClient.exchange_handler.x_rtopic_handler import XRTopicExchangeHandler
```

XRTopicExchangeHandler: Handles x-rtopic exchanges for routing messages based on topics.

Chapter 9

base_message.py

9.1 Class: BaseMessage

Imported by:

```
from EventBusClient.message.base_message import BaseMessage
```

9.1.1 Method: from_data

9.1.2 Method: get_value

Chapter 10

plugin_loader.py

10.1 Class: PluginLoader

Imported by:

```
from EventBusClient.plugin_loader import PluginLoader
```

10.1.1 Method: get_serializer

Get a serializer class by its name.

Arguments:

- `name`

  / *Condition*: required / *Type*: str /
  Name of the serializer class to retrieve.

Returns:

/ *Type*: Serializer | None /
Serializer class or None if not found.

10.1.2 Method: get_exchange_handler

Get an exchange handler class by its name.

Arguments:

- `name`

  / *Condition*: required / *Type*: str /
  Name of the exchange handler class to retrieve.

Returns:

/ *Type*: ExchangeHandler | None /
Exchange handler class or None if not found.

10.1.3 Method: get_message

Get a message class by its name.

Arguments:

- `name`

  / *Condition*: required / *Type*: str /
  Name of the message class to retrieve

Returns:

/ *Type*: BaseMessage | None /
Message class or None if not found.

10.1.4 Method: load_config

Load configuration from a JSONP file located in the same directory as the given config path.

Arguments:

- `config_path`

  / *Condition*: required / *Type*: str /
  Path to the configuration file. If it is a relative path, it will be resolved to an absolute path.

Returns:

/ *Type*: DotDict | None /
A DotDict containing the configuration data if the file exists and is loaded successfully, otherwise None.

Chapter 11

publisher.py

11.1 Class: AsyncPublisher

Imported by:

```
from EventBusClient.publisher import AsyncPublisher
```

AsyncPublisher: Publishes messages to an exchange using aio_pika.

Chapter 12

qlogger.py

12.1 Class: ColorFormatter

Imported by:

```
from EventBusClient.qlogger import ColorFormatter
```

Custom formatter class for setting log color.

12.1.1 Method: format

Set the color format for the log.
Arguments:

- `record`
  / *Condition*: required / *Type*: str /
  Log record.

Returns:

/ *Type*: logging.Formatter /
Log with color formatter.

12.2 Class: QFileHandler

Imported by:

```
from EventBusClient.qlogger import QFileHandler
```

Handler class for user defined file in config.

12.2.1 Method: get_log_path

Get the log file path for this handler.
Arguments:

- `config`
  / *Condition*: required / *Type*: DictToClass /
  Connection configurations.

Returns:

/ *Type*: str /
Log file path.

12.2.2 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
  / *Condition*: required / *Type*: DictToClass /
  Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

12.3 Class: QDefaultFileHandler

Imported by:

```
from EventBusClient.qlogger import QDefaultFileHandler
```

Handler class for default log file path.

12.3.1 Method: get_log_path

Get the log file path for this handler.

Arguments:

- `logger_name`
  / *Condition*: required / *Type*: str /
  Name of the logger.

Returns:

/ *Type*: str /
Log file path.

12.3.2 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
  / *Condition*: required / *Type*: DictToClass /
  Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

12.4 Class: QConsoleHandler

Imported by:

```
from EventBusClient.qlogger import QConsoleHandler
```

Handler class for console log.

12.4.1 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
  / *Condition*: required / *Type*: DictToClass /
  Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

12.5 Class: QLogger

Imported by:

```
from EventBusClient.qlogger import QLogger
```

Logger class for QConnect Libraries.

12.5.1 Method: get_logger

Get the logger object.

Arguments:

- `logger_name`
  / *Condition*: required / *Type*: str /
  Name of the logger.

Returns:

- `logger`
  / *Type*: Logger /
  Logger object. .

12.5.2 Method: set_handler

Set handler for logger.

Arguments:

- `config`
  / *Condition*: required / *Type*: DictToClass /
  Connection configurations.

Returns:

- `handler_ins`
  / *Type*: logging.handler /
  None if no handler is set.
  Handler object.

Chapter 13

base_serializer.py

13.1 Class: Serializer

Imported by:

```
from EventBusClient.serializer.base_serializer import Serializer
```

13.1.1 Method: serialize

Serialize a message object to bytes.

Arguments:

- `msg`

  / *Condition*: required / *Type*: Any /
  Message object to be serialized.

13.1.2 Method: deserialize

Deserialize bytes back into a message object.

Arguments:

- `data`

  / *Condition*: required / *Type*: bytes /
  Serialized message data to be deserialized.

Chapter 14

json_serializer.py

14.1 Class: JsonSerializer

Imported by:

```
from EventBusClient.serializer.json_serializer import JsonSerializer
```

JsonSerializer: Serializes BaseMessage subclasses to JSON strings. Requires message classes to implement: - to_dict() - from_dict(data: dict) eventbusclient-serializer-json-serializer-jsonserializer-serialize ----------

Serialize a message object to JSON bytes.

Arguments:

- `msg`
  / *Condition*: required / *Type*: BaseMessage /
  Message object to be serialized.

Returns:

- bytes
  Serialized message as JSON bytes.

14.1.1 Method: deserialize

Deserialize bytes back into a message object.

Arguments:

- `data`
  / *Condition*: required / *Type*: bytes /
  Serialized message data in bytes format.
- `message_cls`
  / *Condition*: required / *Type*: Type[BaseMessage] /
  Class of the message to deserialize into.

Returns:

/ *Type*: str /
Deserialized message object of type message_cls.

Raises:

- **`ValueError`** If message_cls is not provided.
- **`TypeError`** If message_cls does not implement from_dict(data: dict).
- **`RuntimeError`** If deserialization fails due to invalid data or other issues.

Chapter 15

pickle_serializer.py

15.1 Class: PickleSerializer

Imported by:

```
from EventBusClient.serializer.pickle_serializer import PickleSerializer
```

PickleSerializer: Built-in serializer using Python pickle.

WARNING: Pickle is not secure against untrusted data. Only use in trusted environments. eventbusclient-serializer-pickle-serializer-pickleserializer-serialize ----------

Serialize a message object to bytes using pickle.

Arguments:

- `msg`
  / *Condition*: required / *Type*: Any /
  Message object to be serialized.

15.1.1 Method: deserialize

Deserialize bytes back into a message object using pickle.

Arguments:

- `data`
  / *Condition*: required / *Type*: bytes /
  Serialized message data to be deserialized.

Chapter 16

subscriber.py

16.1 Class: AsyncSubscriber

Imported by:

```
from EventBusClient.subscriber import AsyncSubscriber
```

AsyncSubscriber: Subscribes to messages from an exchange using aio_pika.

Chapter 17

test.py

17.1 Function: run_process

Helper function to run an async function in a process eventbusclient-test-test-main ==========

17.2 Class: TestMessage

Imported by:

```
from EventBusClient.test.test import TestMessage
```

17.2.1 Method: from_data

17.2.2 Method: get_value

Chapter 18

utils.py

18.1 Class: Singleton

Imported by:

```
from EventBusClient.utils import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

> This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

18.2 Class: DictToClass

Imported by:

```
from EventBusClient.utils import DictToClass
```

Class for converting dictionary to class object.

18.2.1 Method: validate

18.3 Class: Utils

Imported by:

```
from EventBusClient.utils import Utils
```

Class to implement utilities for supporting development.

18.3.1 Method: get_all_descendant_classes

Get all descendant classes of a class

Arguments: cls: Input class for finding descendants.

Returns:

> / *Type*: list /
> Array of descendant classes.

18.3.2 Method: get_all_sub_classes

Get all children classes of a class

Arguments:

- `cls`
  / *Condition*: required / *Type*: class /
  Input class for finding children.

Returns:

/ *Type*: list /
Array of children classes.

18.3.3 Method: is_valid_host

18.3.4 Method: caller_name

Get a name of a caller in the format module.class.method

Arguments:

- `skip`
  / *Condition*: required / *Type*: int /

  Specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns:

/ *Type*: str /
An empty string is returned if skipped levels exceed stack height

18.3.5 Method: load_library

Load native library depend on the calling convention.

Arguments:

- `path`
  / *Condition*: required / *Type*: str /
  Library path.
- `is_stdcall`
  / *Condition*: optional / *Type*: bool / *Default*: True /
  Determine if the library's calling convention is stdcall or cdecl.

Returns:

Loaded library object.

18.3.6 Method: is_ascii_or_unicode

Check if the string is ascii or unicode

Arguments: str_check: string for checking codecs: encoding type list

Returns:

/ *Type*: bool /
True : if checked string is ascii or unicode
False : if checked string is not ascii or unicode

18.4 Class: Job

Imported by:

```
from EventBusClient.utils import Job
```

18.4.1 Method: stop

18.4.2 Method: run

18.5 Class: ResultType

Imported by:

```
from EventBusClient.utils import ResultType
```

Result Types.

18.6 Class: ResponseMessage

Imported by:

```
from EventBusClient.utils import ResponseMessage
```

Response message class

18.6.1 Method: get_json

Convert response message to json
Returns:
Response message in json format

18.6.2 Method: get_data

Get string data result
Returns:
String result

18.6.3 Method: create_from_string

Chapter 19

Glossary

RabbitMQ (open-source message broker)
⇒ homepage

EventBusClient (high-level messaging framework built on top of **RabbitMQ**)
⇒ homepage

RobotFramework AIO (**AIO** = **A**ll **I**n **O**ne; test automation framework with extended features, based on the **Robot Framework**)
⇒ RobotFramework AIO homepage
⇒ Robot Framework homepage

Chapter 20

Appendix

About this package:

Table 20.1: Package setup

Setup parameter	Value
Name	EventBusClient
Version	0.1.0
Date	09.07.2025
Description	An IPC message-bus based on RabbitMQ message broker
Package URL	python-rabbitmq-messagebus
Author	Nguyen Huynh Tri Cuong
Email	Cuong.NguyenHuynhTri@vn.bosch.com
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 21

History

0.1.0	05/2022
Initial version	

EventBusClient.pdf
Created at 23.07.2025 - 15:45:58
by GenPackageDoc v. 0.16.0